

Docker

Table of Contents

01-docker-intro

02-dockerfile

03-docker-run

04-docker-comandos

05-docker-avanzado

Docker — Introducción

¿Qué es Docker?

Docker es una plataforma que permite empaquetar una aplicación con todas sus dependencias en una unidad estandarizada llamada **contenedor**. El problema clásico que resuelve es el famoso "en mi máquina sí funciona": al contenerizar la aplicación, esta se comporta igual sin importar dónde se ejecute (local, servidor, nube).

A diferencia de una máquina virtual, un contenedor no incluye un sistema operativo completo. Comparte el kernel del host, lo que lo hace mucho más liviano y rápido de iniciar.

Componentes de Docker

Docker Engine — El corazón de Docker. Es un runtime ligero y conjunto de herramientas que construye y administra contenedores.

Docker CLI (Command Line Interface) — Herramienta que permite al usuario interactuar con Docker directamente mediante comandos. Es donde se ingresan los comandos para interactuar con el demonio de Docker.

Docker Desktop (UI) — Aplicación diseñada para facilitar el flujo de trabajo del desarrollador, proporcionando una interfaz gráfica intuitiva para interactuar con Docker.

Imagen vs Contenedor

	Imagen	Contenedor
Qué es	Plantilla o molde	Instancia en ejecución
Analogía	Receta de cocina	Plato servido
Estado	Inmutable	Modificable efímero
Persistencia	Permanente	Temporal (se pierde al eliminar)

La **imagen** contiene todo lo necesario: sistema base, dependencias, código y configuración. Es estática: una vez creada no cambia.

El **contenedor** es una instancia en ejecución creada a partir de una imagen. Tiene su propia memoria, procesos y sistema de archivos. Puedes crear múltiples contenedores desde una misma imagen.

Flujo de trabajo

1. Escribes el **Dockerfile** con las instrucciones.
2. Ejecutas `docker build` → obtienes una **imagen**.
3. Ejecutas `docker run` → se crea y ejecuta un **contenedor** a partir de esa imagen.

Si modificas el código fuente, la imagen no se actualiza sola. Debes repetir el ciclo: `docker build` para generar una nueva imagen, y `docker run` para crear un nuevo contenedor.

Docker Hub

Docker Hub es el registry público por defecto de Docker. Almacena y distribuye imágenes de contenedores.

Si ejecutas `docker run` o `docker pull` con una imagen que no existe localmente, Docker la busca automáticamente en Docker Hub y la descarga.

BASH

```
docker pull node:22-alpine      # descargar imagen oficial
docker pull nginx:latest        # descargar Nginx
docker tag mi-api usuario/mi-api:v1 # etiquetar para subir
docker push usuario/mi-api:v1     # subir a Docker Hub
```

Las imágenes se versionan con **tags**. `node:22-alpine` significa: imagen `node`, tag `22-alpine` (versión 22 con Alpine). `latest` es el tag por defecto si no se especifica uno.

Licenciamiento

Docker Desktop solía ser gratuito para todos, pero desde agosto de 2021 cambió su modelo de licencia:

- **Docker Desktop** requiere suscripción paga para uso empresarial (empresas con más de 250 empleados o ingresos superiores a \$10M USD).
- **Docker Engine** (el motor de línea de comandos) sigue siendo open source y gratuito bajo la licencia Apache 2.0.
- El uso personal, educativo y en proyectos open source sigue siendo gratuito en Docker Desktop.

Opciones gratuitas

- Usar solo Docker Engine (CLI) sin Docker Desktop.
- Usar alternativas open source como Podman o Colima.

Alternativas Open Source

Podman

Podman es un motor de contenedores open source, diseñado como **drop-in replacement** de Docker. Desarrollado por Red Hat.

- Soporta Dockerfile y puede ejecutar imágenes de Docker sin conversión.
- **No requiere un daemon central:** cada contenedor se ejecuta como un proceso hijo directo, lo que mejora seguridad y simplicidad.
- **No necesita privilegios de root** para ejecutar contenedores.
- CLI casi idéntica a Docker (puedes usar `alias docker=podman`).
- Compatible con Docker Compose mediante `podman-compose`.

BASH

```
podman build -t mi-api .
podman run -d -p 3000:3000 mi-api
podman ps
```

Colima

Colima es una alternativa ligera a Docker Desktop para macOS. Ejecuta contenedores usando Lima (máquinas virtuales) bajo el capó.

- No requiere Docker Desktop.
- Open source y gratuito.
- Compatible con la CLI de Docker.
- Soporta arquitectura ARM e Intel.

BASH

```
colima start      # inicia la VM
colima stop       # detiene la VM
```

Otras alternativas

- **containerd** — runtime de contenedores usado internamente por Docker, también se puede usar directamente.
- **nerdctl** — CLI compatible con Docker para containerd.
- **Rancher Desktop** — alternativa multiplataforma que usa containerd y Kubernetes.

Extensiones de VSCode

- **Docker** (Microsoft) — administrar contenedores, imágenes, Dockerfiles y Compose directamente desde el IDE.

- **Dev Containers** (Microsoft) — desarrollar dentro de contenedores usando VSCode remoto.

Docker — Dockerfile

¿Qué es un Dockerfile?

Un Dockerfile es un archivo de texto que contiene las instrucciones para construir una **imagen** de Docker. Esa imagen es una plantilla reutilizable que incluye todo lo necesario para ejecutar una aplicación: sistema operativo base, dependencias, librerías, archivos de código y comandos de inicio.

A continuación se explica cada instrucción de un Dockerfile típico para una aplicación Node.js.

FROM — Imagen base

```
DOCKERFILE
```

```
FROM node:22-alpine
```

Toda imagen parte de una **imagen base**. En este caso usamos `node:22-alpine`, una imagen oficial de Node.js versión 22. El sufijo `alpine` indica que utiliza Alpine Linux, una distribución extremadamente liviana que mantiene la imagen pequeña y eficiente. Esto se traduce en builds más rápidos y menos espacio en disco.

WORKDIR — Directorio de trabajo

```
DOCKERFILE
```

```
WORKDIR /app
```

Define el **directorio de trabajo** dentro del contenedor. Si la carpeta no existe, Docker la crea automáticamente. Todas las instrucciones posteriores (`COPY`, `RUN`, `CMD`) se ejecutarán dentro de esta carpeta. No es necesario que tu proyecto local tenga una carpeta llamada `app`; esto es interno del contenedor.

COPY — Copiar archivos al contenedor

```
DOCKERFILE
```

```
COPY package*.json .
```

Copia `package.json` y `package-lock.json` del host al directorio de trabajo del contenedor. El `.` al final significa "aquí, en el WORKDIR actual". Es buena práctica

copiar primero solo los archivos de dependencias para aprovechar el **sistema de capas** de Docker: si no cambian, esta capa se reutiliza de la caché en builds posteriores.

RUN — Ejecutar comandos durante el build

DOCKERFILE

```
RUN npm install
```

Ejecuta un comando dentro del contenedor durante la construcción de la imagen. En este caso instala todas las dependencias definidas en `package.json`. El `node_modules` resultante queda dentro del contenedor, aislado del sistema host.

COPY — Copiar el resto del proyecto

DOCKERFILE

```
COPY . .
```

Copia el resto de los archivos del proyecto (código fuente, rutas, controladores, etc.) al contenedor. Se hace después de instalar dependencias para que los cambios en el código no invaliden la caché de `npm install`.

EXPOSE — Exponer puertos

DOCKERFILE

```
EXPOSE 3000
```

Documenta que la aplicación usará el puerto 3000. Es una declaración informativa: no publica el puerto por sí sola. Sirve como documentación para quien use la imagen y como referencia para el mapeo de puertos en tiempo de ejecución.

La publicación real del puerto se hace al ejecutar el contenedor con `-p`.

CMD — Comando de inicio

DOCKERFILE

```
CMD ["npm", "start"]
```

Define el comando que se ejecutará cuando el contenedor se inicie. Cada palabra del comando va como un elemento separado del arreglo (formato exec). También se podría escribir como `CMD npm start` (formato shell), pero el formato exec es más predecible y seguro.

Dockerfile completo

DOCKERFILE

```
FROM node:22-alpine

WORKDIR /app

COPY package*.json .

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

Construir la imagen

docker build

`docker build` procesa el Dockerfile y genera una imagen. Lee las instrucciones línea por línea y crea una **capa** por cada instrucción, lo que permite cachear y reutilizar capas inmutables en builds futuros.

BASH

```
docker build -t mi-api .
```

Desglose del comando:

- `docker build` — construye la imagen a partir de un Dockerfile.
- `-t mi-api` — asigna un nombre (tag) a la imagen. Sin tag, la imagen se identifica solo por un ID.
- `.` — el **contexto de construcción**. Docker empaqueta todo el contenido del directorio actual y lo envía al demonio para construir la imagen. Por eso es importante tener un `.dockerignore` para excluir archivos innecesarios.

Caché de capas

Cada instrucción del Dockerfile genera una capa. Docker cachea cada capa y si una instrucción no ha cambiado entre builds, reutiliza la capa cacheada. Por eso se copia `package*.json` antes que el resto del código: si solo cambias el código fuente, `npm install` no se vuelve a ejecutar.

Ver las imágenes disponibles

BASH

```
docker images
```

TEXT

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
mi-api	latest	a1b2c3d4e5f6	2 minutes ago	180MB

`docker images` lista todas las imágenes almacenadas localmente, incluyendo las que construiste y las que descargaste de registries como Docker Hub.

.dockerignore

Funciona como `.gitignore` : excluye archivos del **contexto de build** para que no se envíen al demonio de Docker. Esto acelera el build y reduce el tamaño de la imagen final.

DOCKERIGNORE

```
node_modules
.git
.env
.env.local
*.log
dist
.vscode
.DS_Store
```

Cada línea es un patrón de archivo o carpeta que Docker ignorará al construir.

Docker — Ejecutar contenedores

docker run

`docker run` crea un contenedor a partir de una imagen y lo ejecuta.

BASH

```
docker run mi-api
```

Lo que Docker hace internamente:

1. Busca la imagen `mi-api` localmente. Si no existe, la descarga de Docker Hub.
2. Crea un contenedor nuevo a partir de esa imagen.
3. Ejecuta el comando definido en `CMD` (ej. `npm start`).
4. La aplicación empieza a correr dentro del contenedor.

Principales flags

-d (detached)

BASH

```
docker run -d mi-api
```

Ejecuta el contenedor en **segundo plano**. Sin `-d`, los logs se muestran en la terminal y esta queda ocupada. Con `-d`, recuperas el control inmediatamente.

-p (port mapping)

BASH

```
docker run -p 3000:3000 mi-api
```

Formato: `-p <puerto_host>:<puerto_contenedor>`

Conecta un puerto de tu computadora al puerto del contenedor. El contenedor tiene su propia red interna y no es accesible directamente desde el host. El mapeo crea un puente:

- `-p 3000:3000` → accedes desde `http://localhost:3000`

- `-p 8080:3000` → el contenedor sigue usando el 3000, pero accedes desde `http://localhost:8080`

`--name`

BASH

```
docker run --name api mi-api
```

Asigna un nombre personalizado al contenedor. Si no se asigna, Docker genera uno aleatorio como `happy_einstein`. El nombre facilita administrarlo (logs, stop, rm).

`--rm`

BASH

```
docker run --rm -p 5002:3000 02-node-web
```

Elimina el contenedor automáticamente cuando se detiene. Ideal para pruebas rápidas donde no necesitas conservar el contenedor después de usarlo.

`-e` (environment variables)

BASH

```
docker run -d -p 5005:3000 -e SALUDO="Hola" --name web-env 02-node-web
```

Inyecta **variables de entorno** al contenedor. Útil para pasar configuraciones como API keys, URLs de base de datos o entornos (`NODE_ENV`).

`--restart`

BASH

```
docker run -d --restart always --name api mi-api
```

Define la política de reinicio. Opciones comunes:

- `no` — no reiniciar (default).
- `always` — reiniciar siempre que se detenga.
- `on-failure` — reiniciar solo si termina con error.
- `unless-stopped` — reiniciar a menos que lo detengas explícitamente.

Comando completo típico

BASH

```
docker run -d -p 3000:3000 --name api mi-api
```

Significa: "Crea un contenedor llamado `api` , usando la imagen `mi-api` , ejecútalo en segundo plano y conecta el puerto 3000 del contenedor con el puerto 3000 de mi computadora."

Docker — Comandos esenciales

docker ps

BASH

```
docker ps
```

Lista los contenedores **en ejecución**. Muestra ID, nombre, imagen, estado, puertos mapeados y tiempo activo.

BASH

```
docker ps -a
```

Lista **todos** los contenedores, incluyendo los detenidos. Útil para encontrar contenedores que ya terminaron pero siguen ocupando espacio.

BASH

```
docker ps -q
```

Muestra solo los IDs numéricos. Útil para usar en combinación con otros comandos (ej. `docker stop $(docker ps -q)`).

docker stop / start

BASH

```
docker stop <id_o_nombre>
```

Detiene un contenedor de forma controlada (envía SIGTERM y espera unos segundos antes de forzar el cierre).

BASH

```
docker start <id_o_nombre>
```

Inicia un contenedor que estaba detenido. Los datos dentro del contenedor se conservan.

docker rm

BASH

```
docker rm <id_o_nombre>
```

Elimina un contenedor. Debe estar detenido primero.

BASH

```
docker rm -f <id_o_nombre>
```

Fuerza la eliminación incluso si el contenedor está en ejecución.

BASH

```
docker container prune
```

Elimina todos los contenedores detenidos de una sola vez.

docker logs

BASH

```
docker logs <id_o_nombre>
```

Muestra los logs (salida estándar y de error) de un contenedor.

BASH

```
docker logs -f <id_o_nombre>
```

Muestra los logs en **tiempo real** (follow), similar a `tail -f`. Útil para debug mientras la app está corriendo.

BASH

```
docker logs --tail 50 <id_o_nombre>
```

Muestra solo las últimas 50 líneas.

docker exec

BASH

```
docker exec -it <id_o_nombre> sh
```

Ejecuta un comando **dentro de un contenedor que ya está corriendo**. Es como abrir una terminal dentro del contenedor.

- `-i` — interactivo (mantiene STDIN abierto).
- `-t` — asigna un pseudo-TTY (terminal).
- `sh` — abre una shell. También funciona `bash` si la imagen lo tiene.

Útil para inspeccionar archivos, variables de entorno o procesos dentro del contenedor.

BASH

```
docker exec <id> cat /app/.env
docker exec <id> ls -la
```

docker images

BASH

```
docker images
```

Lista las imágenes disponibles localmente (las que construiste y las descargadas).

BASH

```
docker rmi <imagen_id>
```

Elimina una imagen. Si hay contenedores que la usan, debes eliminarlos primero.

docker pull / push

BASH

```
docker pull node:22-alpine
```

Descarga una imagen de Docker Hub (o cualquier registry configurado) sin ejecutarla.

BASH

```
docker push usuario/mi-api
```

Sube una imagen a un registry (Docker Hub, AWS ECR, etc.). La imagen debe estar etiquetada con el nombre del registry.

docker inspect

BASH

```
docker inspect <id_o_nombre>
```

Muestra información detallada de un contenedor o imagen en formato JSON: configuración, red, variables de entorno, montajes, etc.

docker system

BASH

```
docker system df
```

Muestra el uso de espacio en disco de Docker: imágenes, contenedores, volúmenes y build cache. Útil para diagnosticar por qué Docker ocupa tanto espacio.

TEXT

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	5	2	2.1GB	1.3GB (62%)
Containers	8	1	50MB	45MB (90%)
Local Volumes	3	1	500MB	200MB (40%)
Build Cache	-	-	300MB	300MB

BASH

```
docker system prune
```

Elimina contenedores detenidos, redes no usadas, imágenes colgantes y build cache.

BASH

```
docker system prune -a
```

Más agresivo: también elimina imágenes **no usadas** (no solo las colgantes). Libera la mayor cantidad de espacio posible.

Docker — Avanzado

Volúmenes

Los contenedores son efímeros: cuando se eliminan, los datos que contienen se pierden. Los **volúmenes** permiten persistir datos fuera del ciclo de vida del contenedor.

Tipos de montajes

Named volumes — administrados por Docker:

BASH

```
docker run -v mi-volumen:/app/data mi-api
```

Docker crea y gestiona el volumen. Los datos persisten incluso si eliminas el contenedor.

Bind mounts — carpeta del host montada directamente:

BASH

```
docker run -v $(pwd):/app mi-api
```

Monta el directorio actual del host en `/app` del contenedor. Útil para desarrollo porque los cambios locales se reflejan al instante.

tmpfs mounts — en memoria (solo Linux):

BASH

```
docker run --tmpfs /app/temp mi-api
```

Datos temporales que se pierden al detener el contenedor.

docker volume

BASH

```
docker volume ls
docker volume create mi-volumen
docker volume rm mi-volumen
docker volume prune
```

Redes

Cada contenedor tiene su propia interfaz de red. Docker ofrece varios **drivers de red** para controlar cómo se comunican los contenedores entre sí y con el exterior.

Drivers

bridge (default) Crea una red privada interna. Los contenedores en la misma red bridge pueden comunicarse entre sí por nombre. Aislados de contenedores en otras bridges.

BASH

```
docker network create mi-red
docker run -d --net mi-red --name api mi-api
docker run -d --net mi-red --name db postgres
# api puede conectarse a db usando el nombre "db"
```

host El contenedor usa la red del host directamente. No hay aislamiento de red. Útil cuando necesitas máximo rendimiento de red.

none Sin red. Contenedor completamente aislado.

Comandos de red

BASH

```
docker network ls          # listar redes
docker network create mi-red # crear red personalizada
docker network connect mi-red api # conectar contenedor a una red
docker network inspect mi-red # ver detalles
docker network prune       # eliminar redes no usadas
```

Docker Compose

Cuando una aplicación tiene múltiples servicios (API, base de datos, caché, cola de mensajes), manejarlos individualmente con `docker run` se vuelve tedioso. **Docker Compose** permite definir y ejecutar aplicaciones multi-contenedor con un solo archivo YAML.

docker-compose.yml

YAML

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DB_HOST=db
    depends_on:
      - db

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: mi-app
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Comandos de Compose

BASH

<code>docker compose up -d</code>	<code># levantar todos los servicios</code>
<code>docker compose down</code>	<code># detener y eliminar contenedores/redes</code>
<code>docker compose down -v</code>	<code># lo mismo + elimina volúmenes</code>
<code>docker compose logs -f</code>	<code># logs de todos los servicios</code>
<code>docker compose logs -f api</code>	<code># logs de un servicio específico</code>
<code>docker compose ps</code>	<code># estado de los servicios</code>
<code>docker compose exec api sh</code>	<code># ejecutar comando en un servicio</code>
<code>docker compose build</code>	<code># reconstruir imágenes</code>
<code>docker compose up -d --build</code>	<code># reconstruir y levantar</code>